

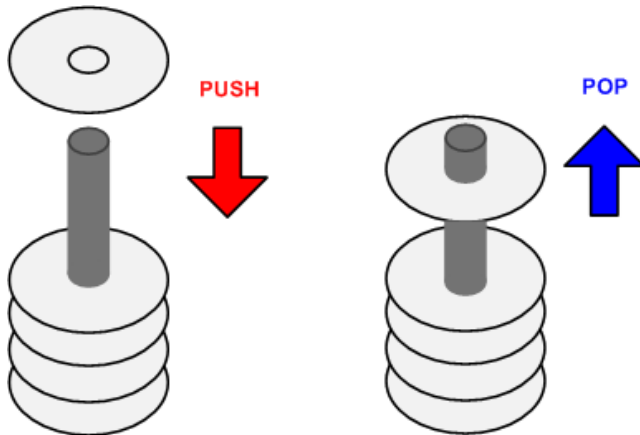
Embedded Systems with ARM Cortex-M Microcontrollers in Assembly Language and C

CSE33 I: Microprocessor Interfacing and Embedded Systems

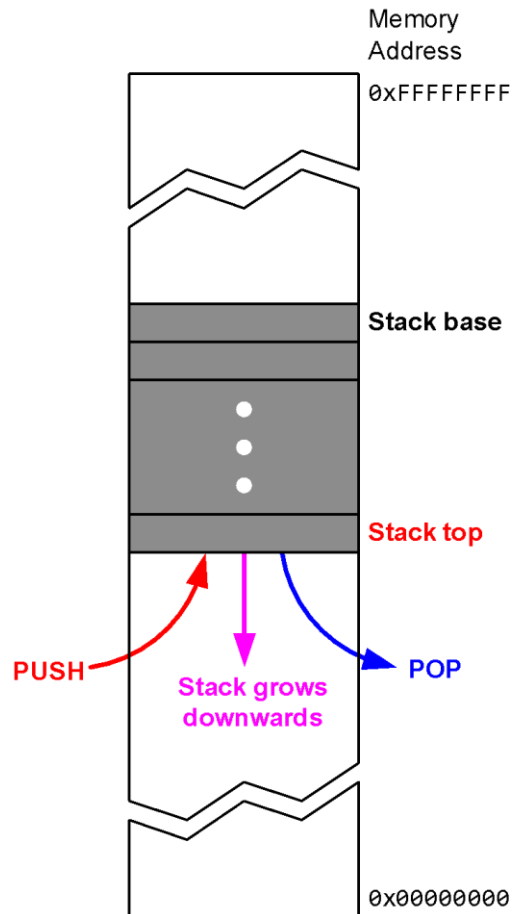
Lecture#8: Stack & Recursion (Chapter#8)

Stack

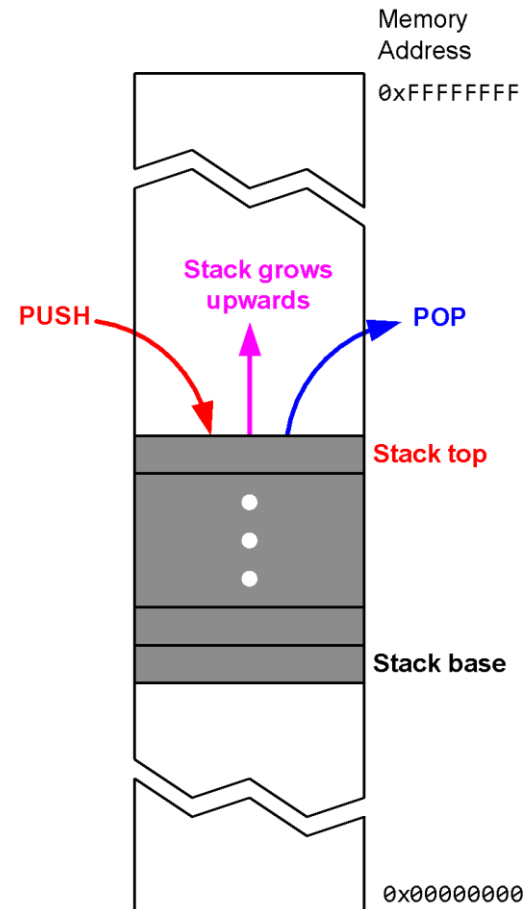
- ❑ A **Last-In-First-Out** data structure
- ❑ Only allow to access the most recently added item
 - ❑ Also called the top of the stack
- ❑ Key operations:
 - ❑ push (add item to stack)
 - ❑ pop (remove top item from stack)



Stack Growth Convention: Ascending *vs* Descending



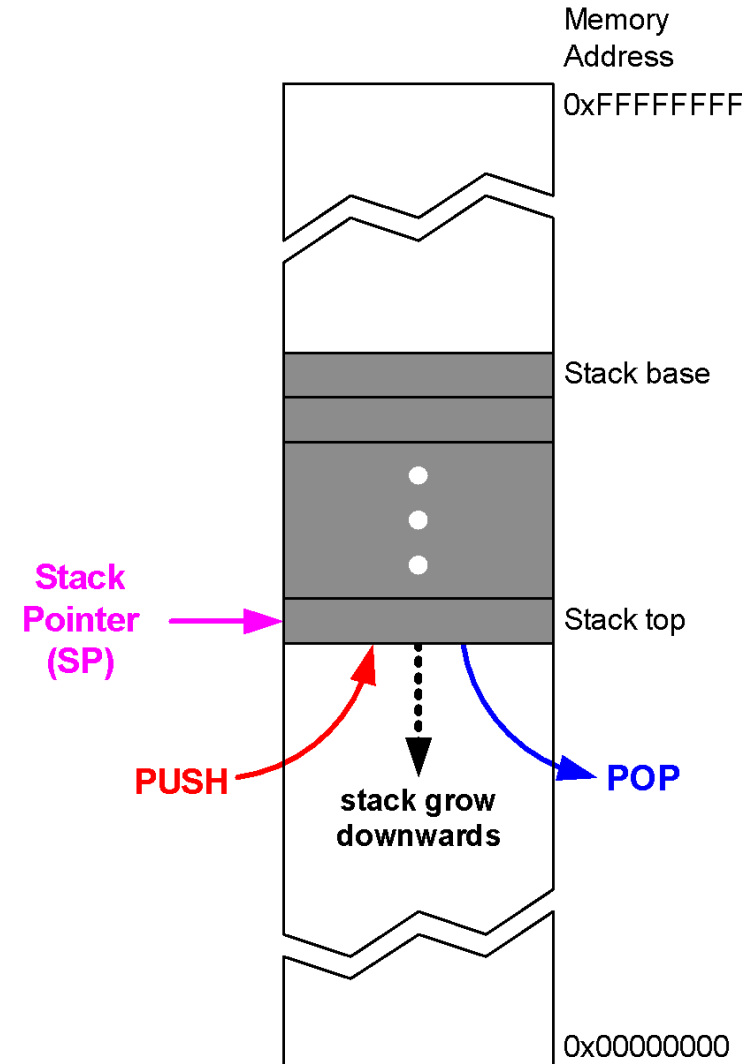
Descending stack. Stack grows towards low memory address



Ascending stack. Stack grows towards high memory address

Cortex-M Stack

- ❑ stack pointer (SP) = R13
- ❑ Cortex-M uses *full descending stack*
- ❑ stack pointer
 - ❑ decremented on **PUSH**
 - ❑ incremented on **POP**
 - ❑ SP starts at **0x20000200** for STM32-Discovery



Stack

PUSH {*Rd*}

❑ $SP = SP - 4 \rightarrow$ descending stack

❑ $(*SP) = Rd \rightarrow$ full stack (r8, r7, r6)

Push multiple registers

They are equivalent.

PUSH {r6, r7, r8} $\longleftrightarrow$ **PUSH {r7, r6, r8}** $\longleftrightarrow$ **PUSH {r8}**
PUSH {r7}
PUSH {r6}

- The order in which registers listed in the register list does not matter.
- When pushing multiple registers, these registers are automatically **sorted by name** and **the lowest-numbered register** is stored to the lowest memory address, *i.e.* **is stored last**.

Stack

POP {Rd}

- ❑ $Rd = (*SP) \rightarrow$ full stack
- ❑ $SP = SP + 4 \rightarrow$ Stack shrinks

Pop multiple registers

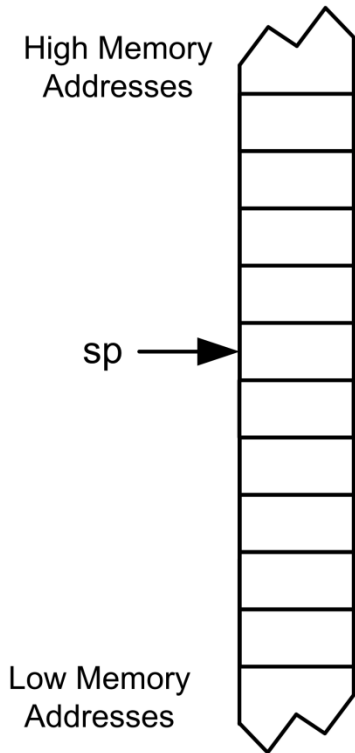
They are equivalent.

POP {r6, r7, r8} $\longleftrightarrow$ POP {r8, r6, r7} $\longleftrightarrow$
 POP {r6}
 POP {r7}
 POP {r8}

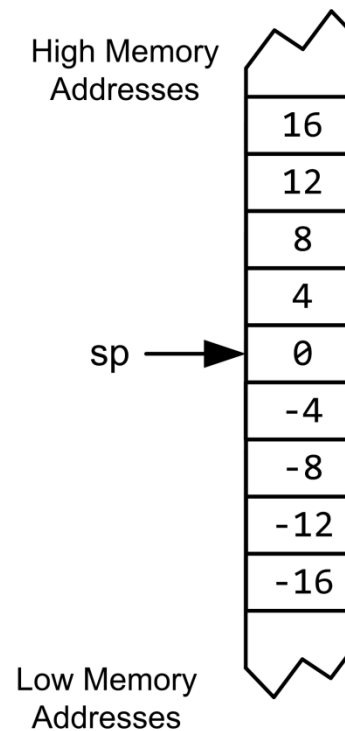
- The order in which registers listed in the register list does not matter.
- When popping multiple registers, these registers are automatically **loaded by name** and **the lowest-numbered register** is loaded from the lowest memory address, *i.e.* **is loaded first**.

Full Descending Stack

PUSH {r1, r3, r2, r7}

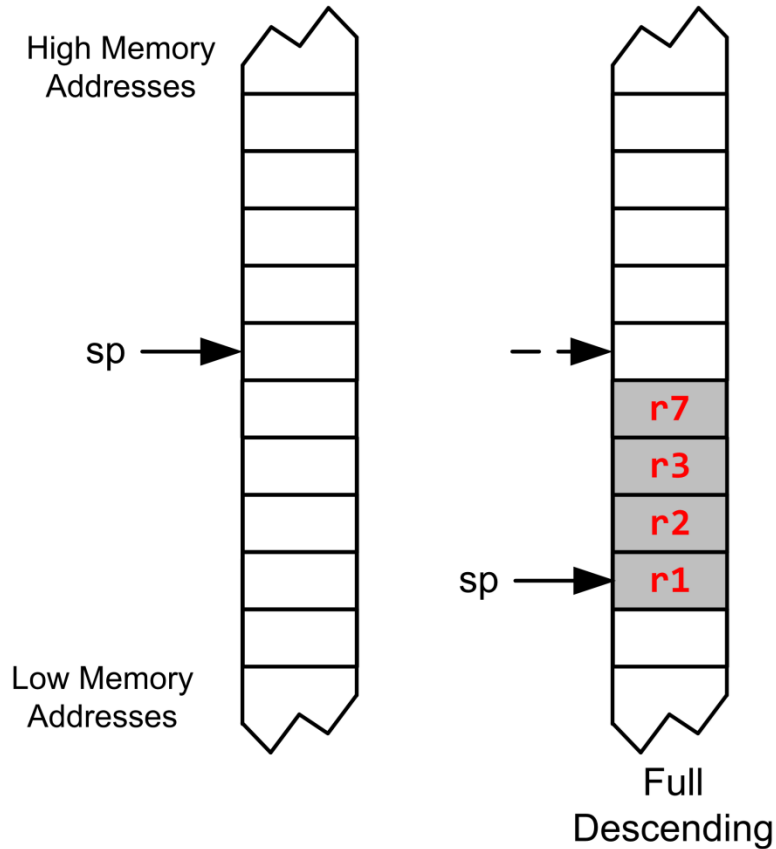


POP {r3, r1, r7, r2}

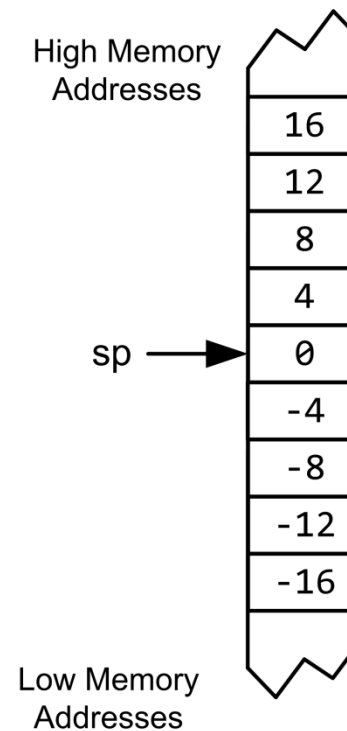


Stack

PUSH {r3, r1, r7, r2}

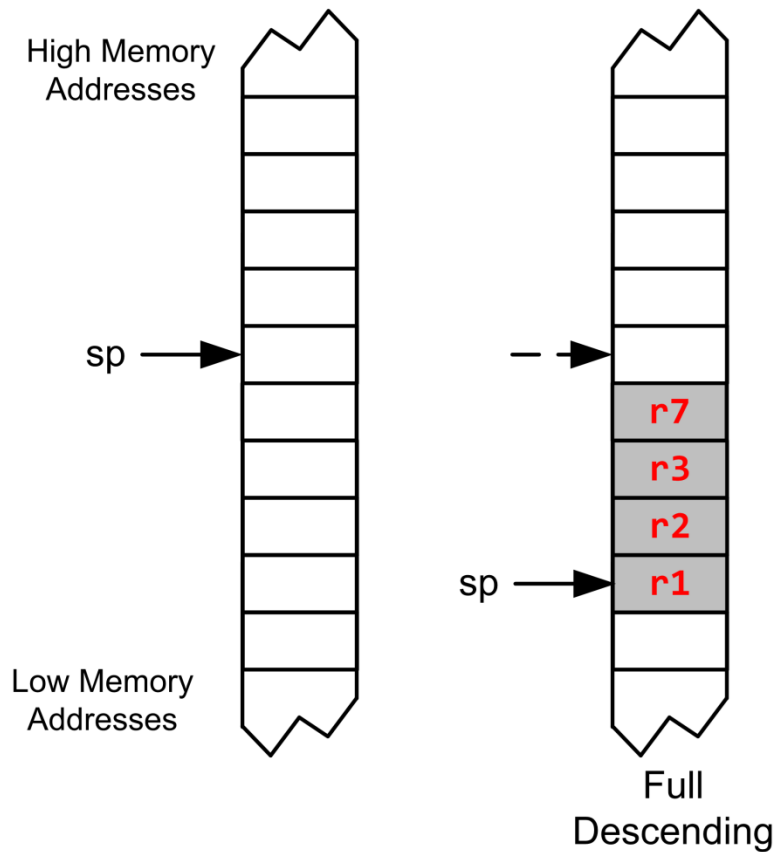


POP {r3, r1, r7, r2}

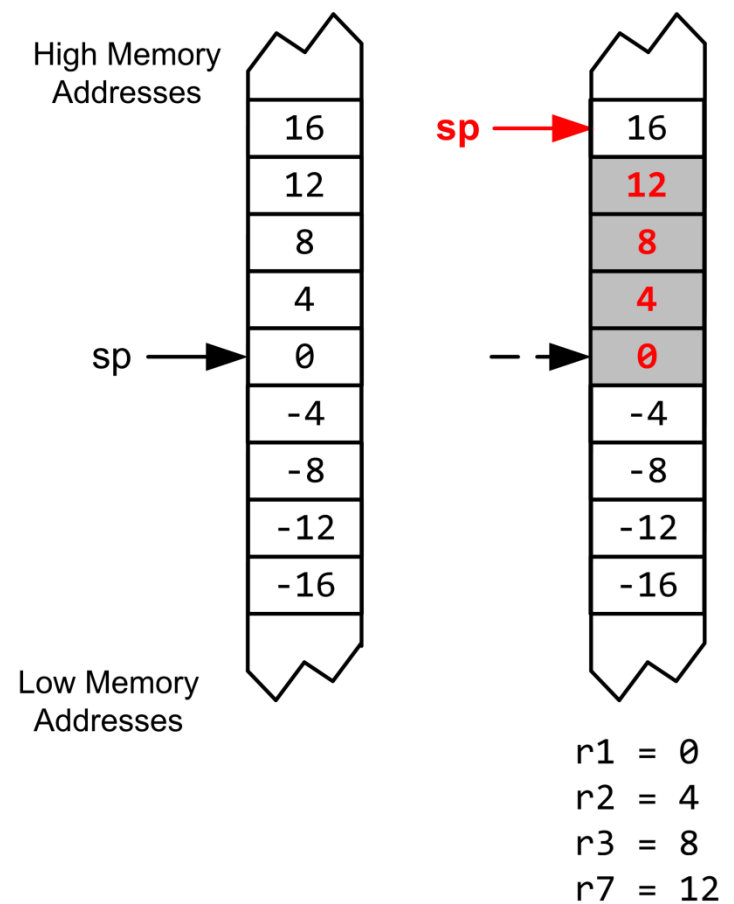


Stack

PUSH {r1, r2, r3, r7}



POP {r3, r1, r7, r2}



Recursive Procedure Example

- C code:

```
int fact (int n)
{
    if (n < 1) return 1;
    else return n * fact(n - 1);
}
```

Recursive Functions

- ▣ **push LR** (& working registers) onto stack before nested call
- ▣ **pop LR** (& working registers) off stack after nested return

Recursive Factorial in Assembly

.global main

main:

MOV r0, #0x03

BL factorial

B stop

factorial:

PUSH {r4, lr}

MOV r4, r0

CMP r4, #0x01

BNE NZ

MOVS r0, #0x01

loop: POP {r4, pc}

NZ: SUBS r0, r4, #1

BL factorial

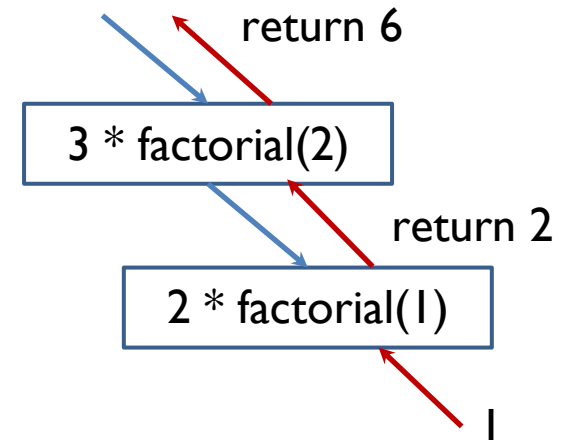
MUL r0, r4, r0

B loop

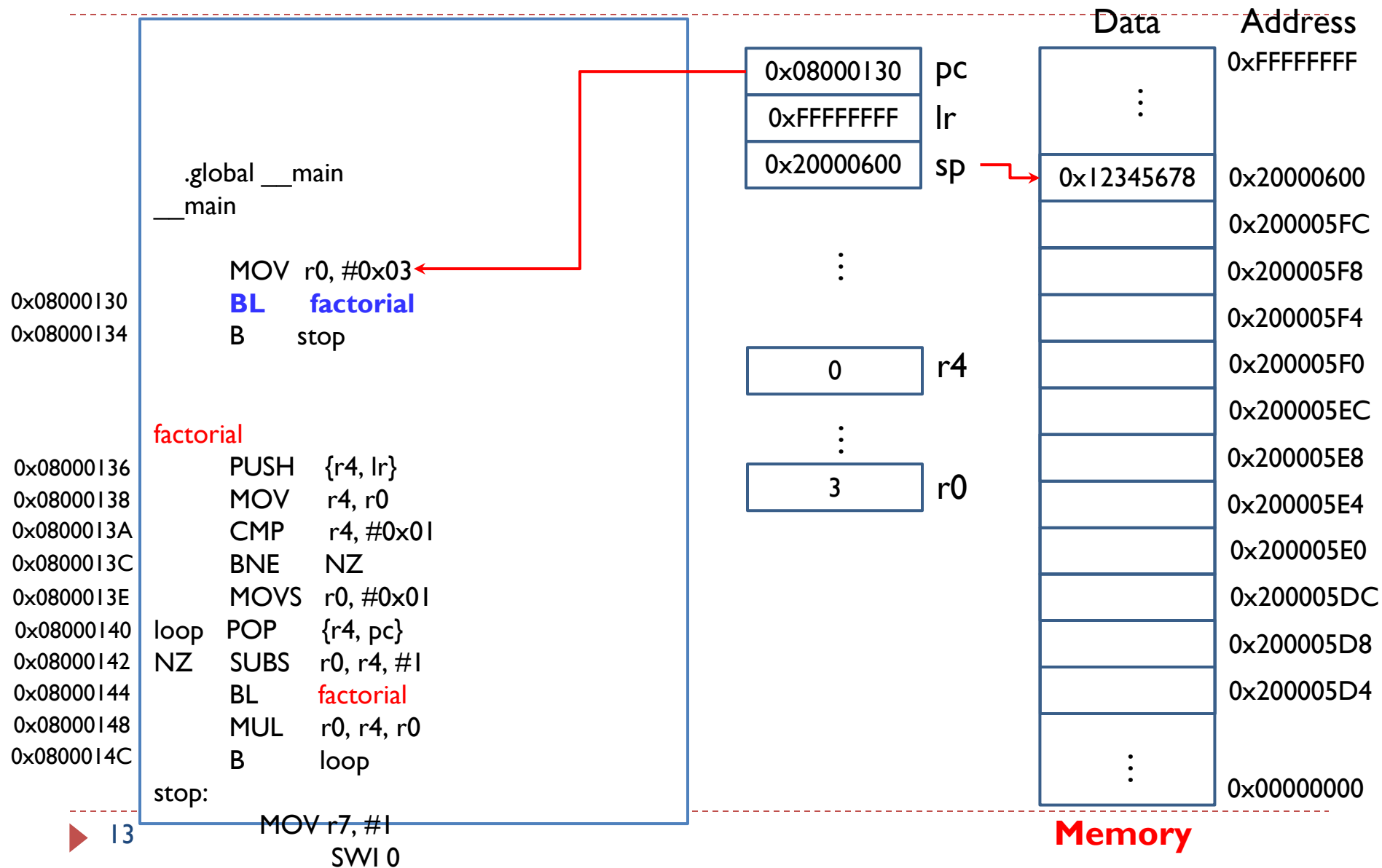
stop:

MOV r7, #1

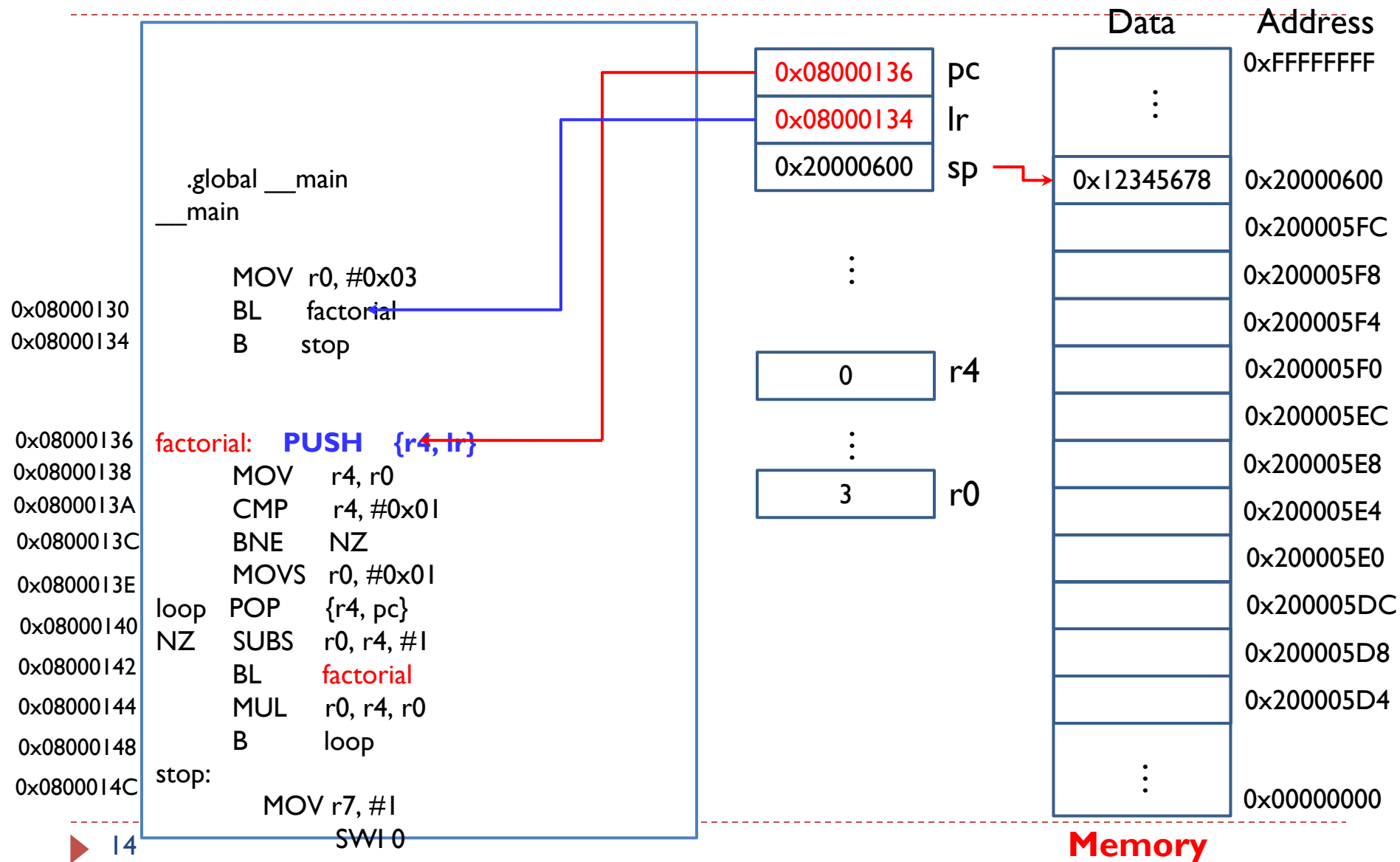
SWI 0



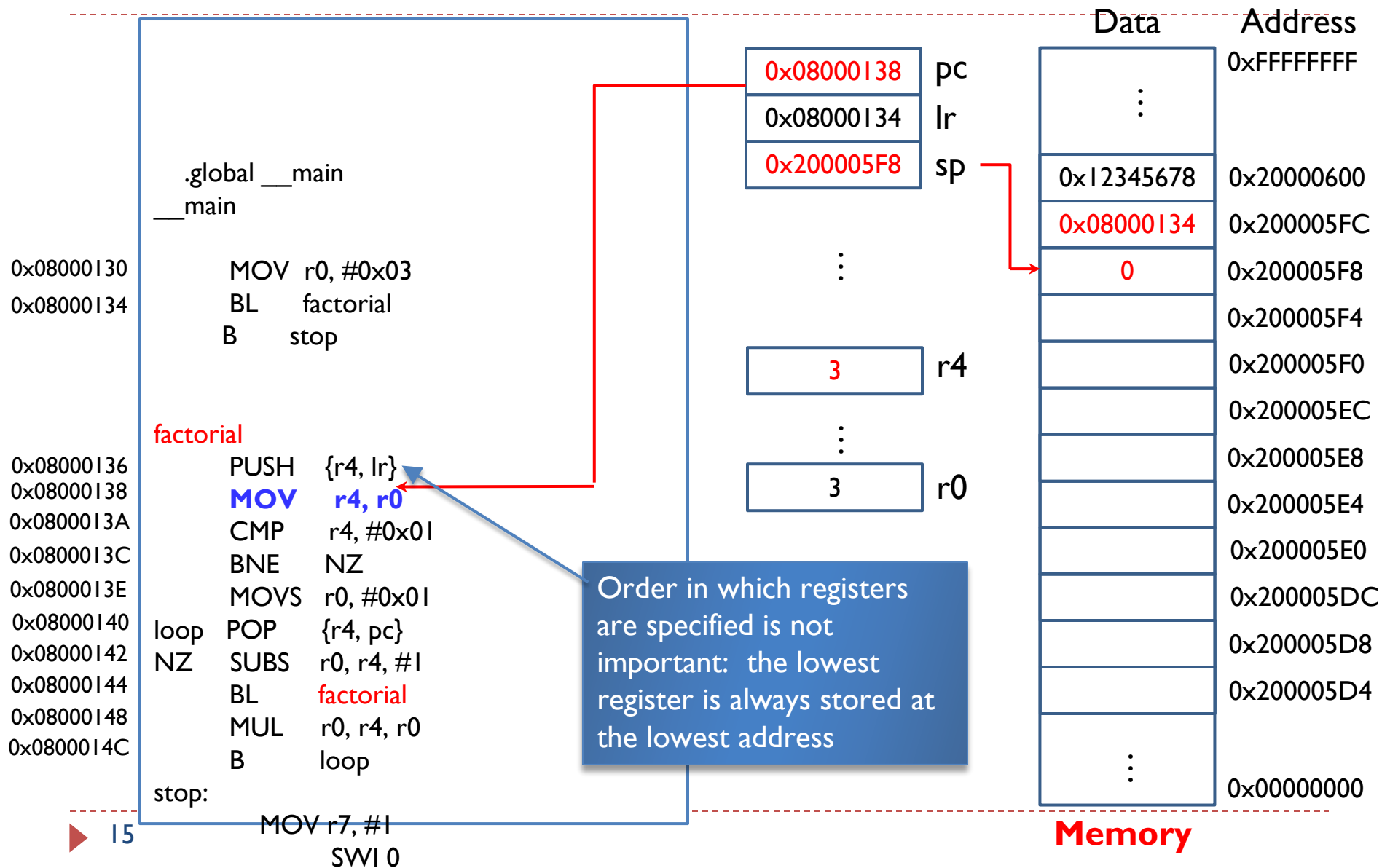
Recursive Factorial in Assembly



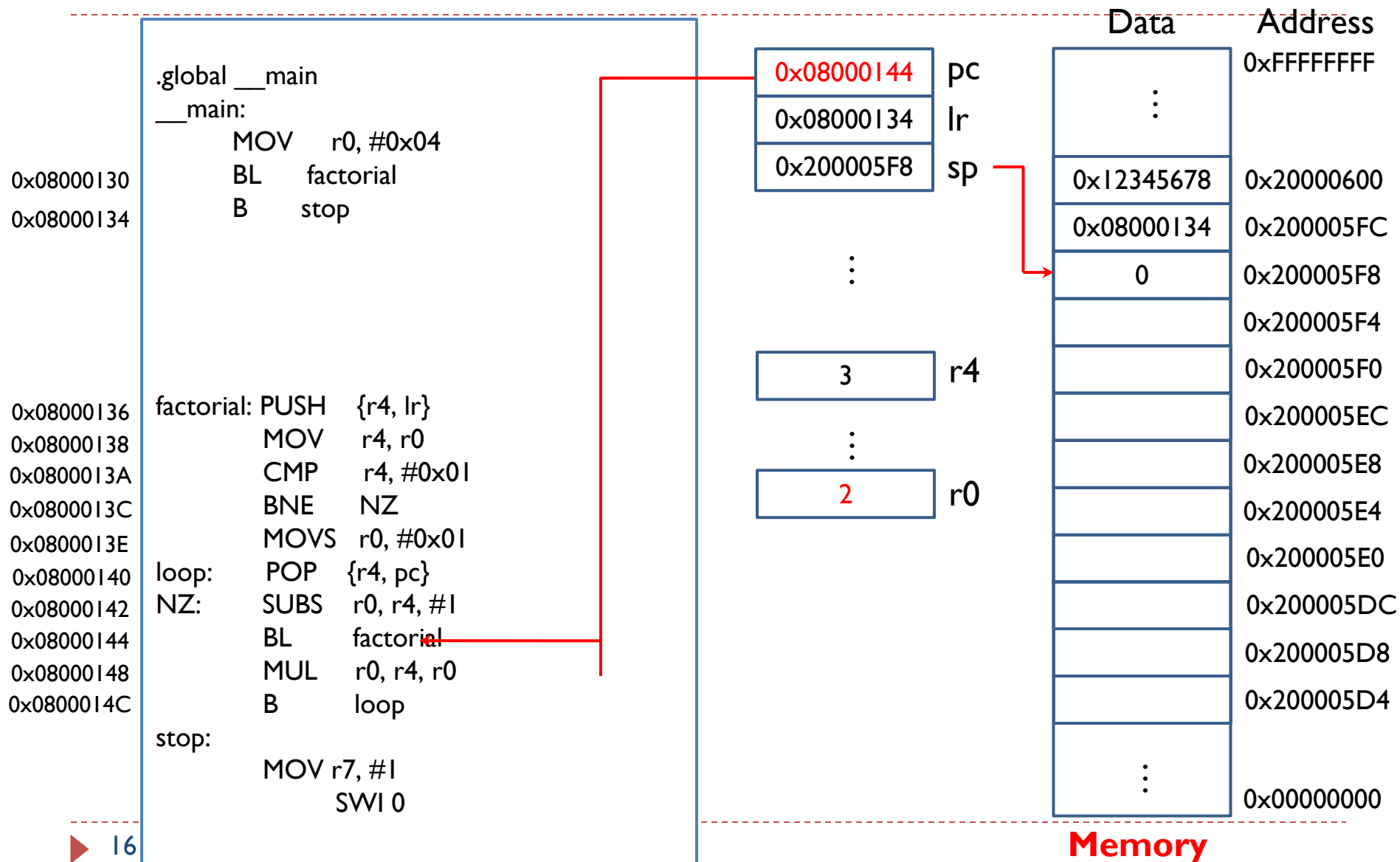
Recursive Factorial in Assembly



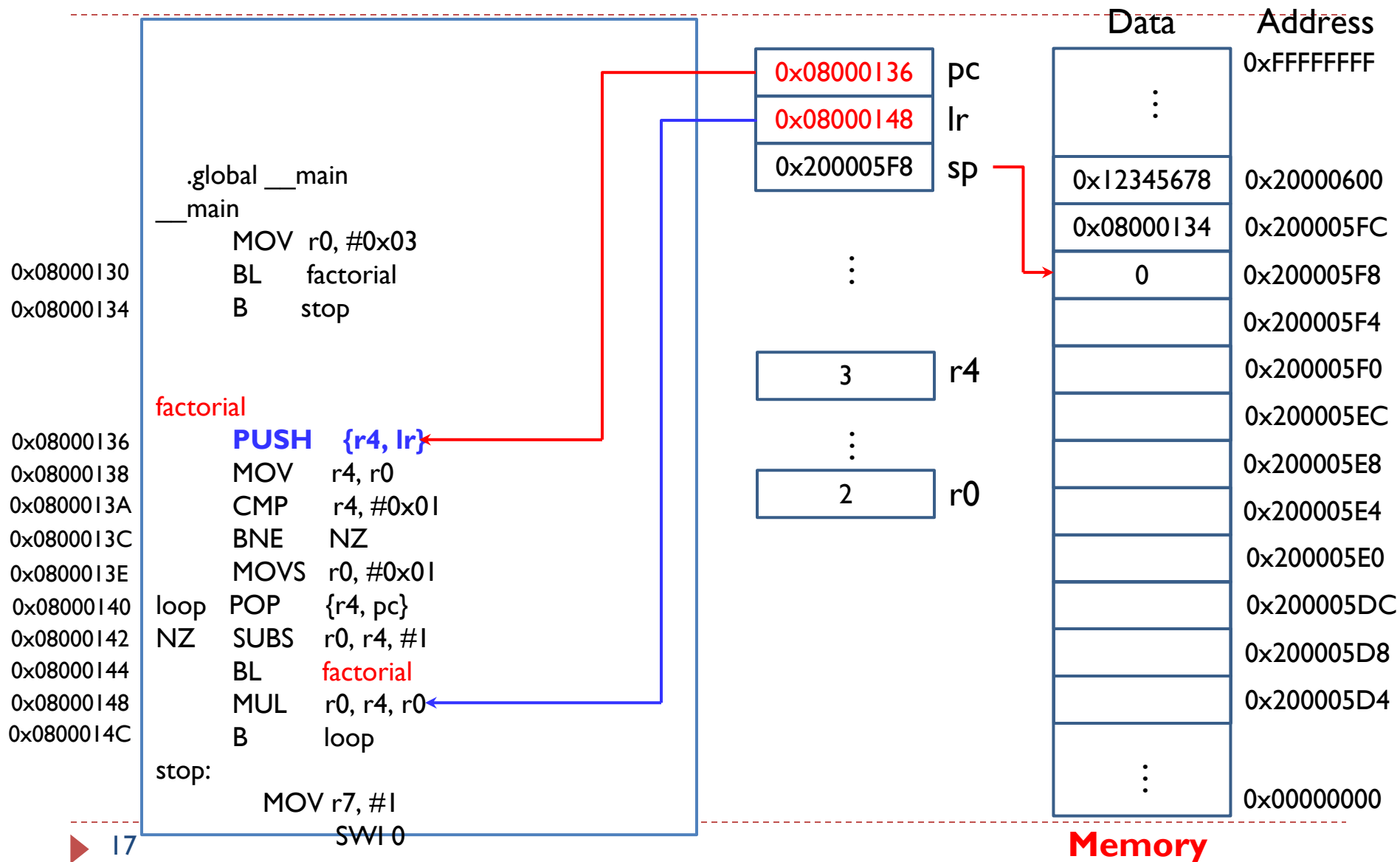
Recursive Factorial in Assembly



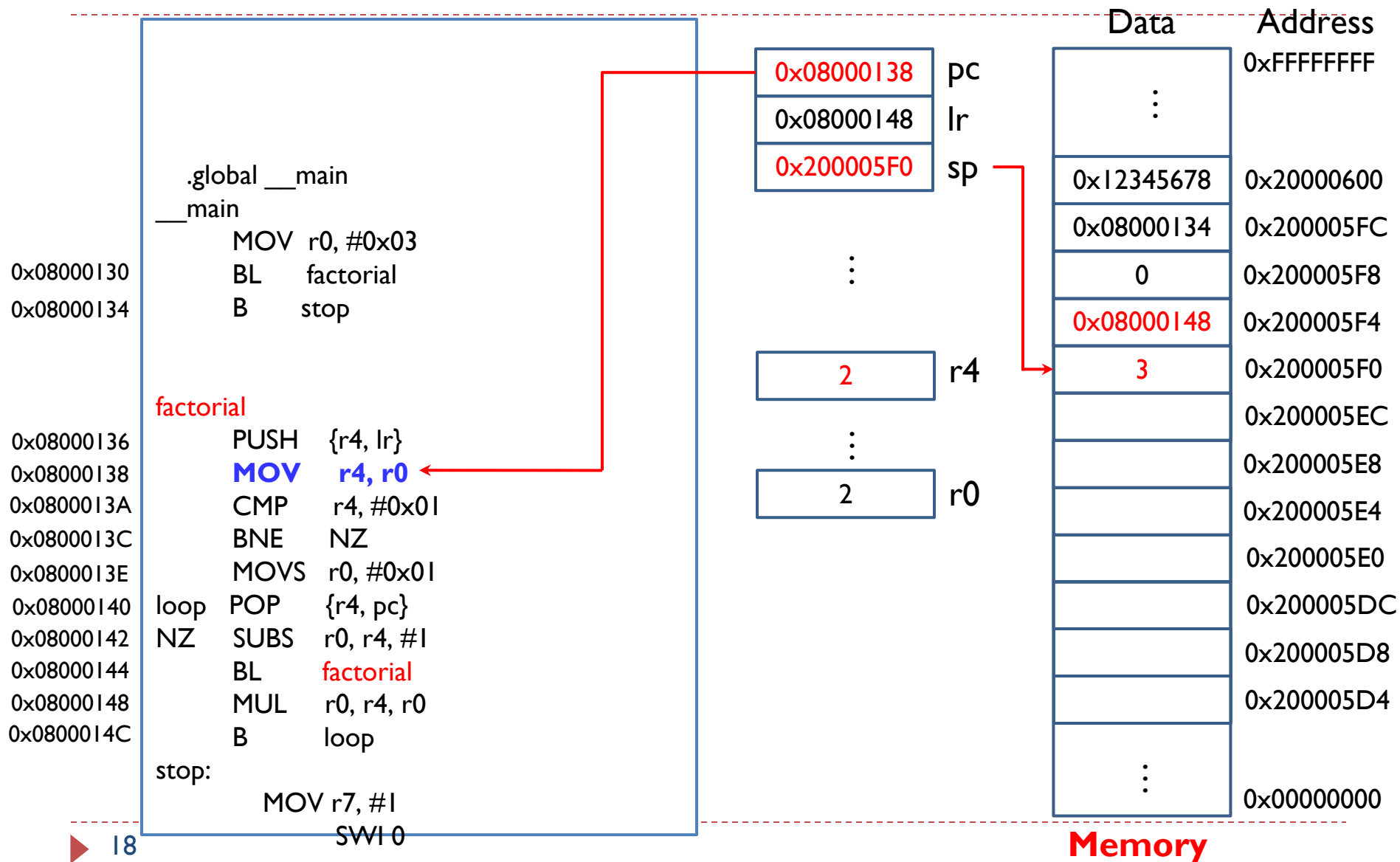
Recursive Factorial in Assembly



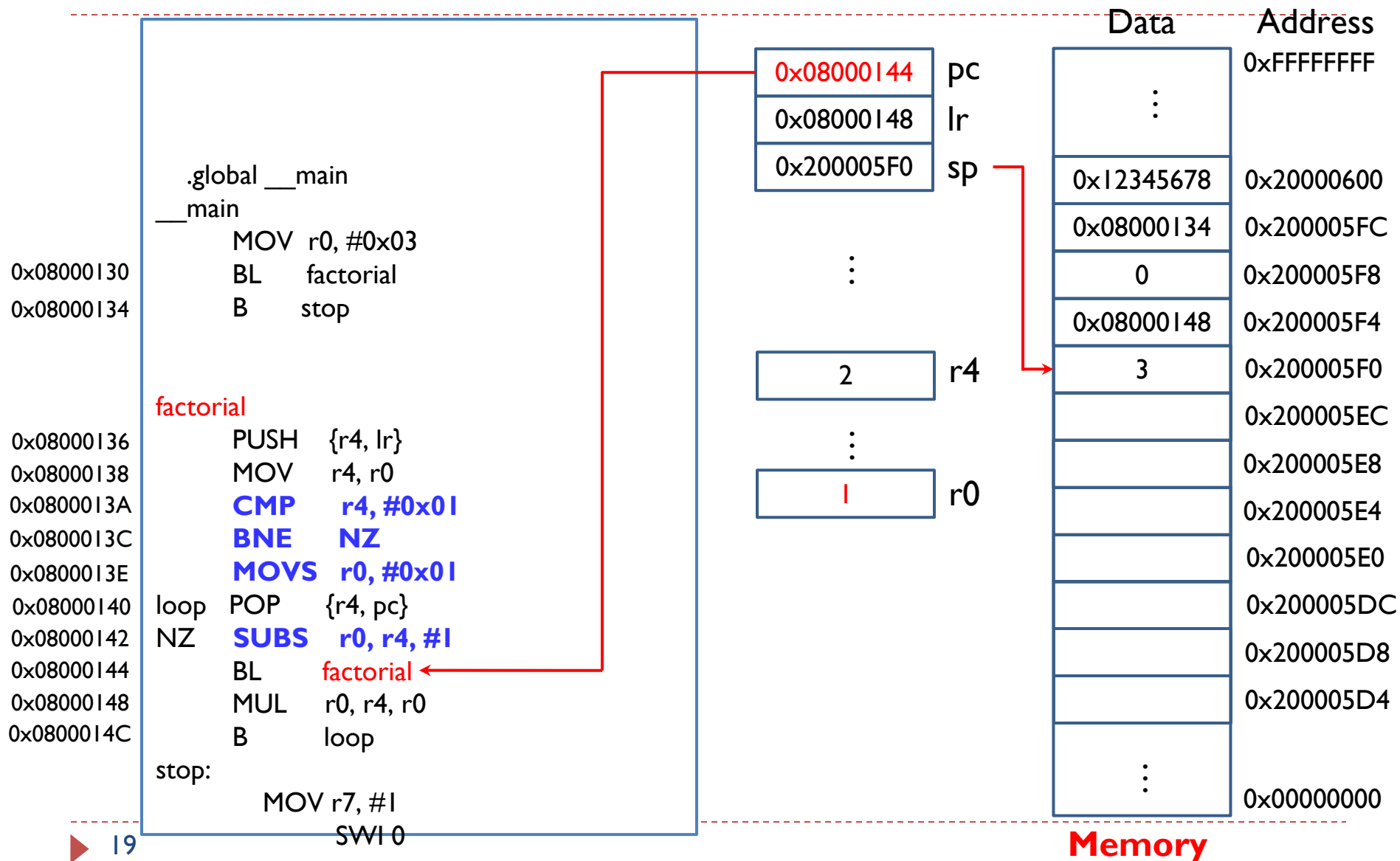
Recursive Factorial in Assembly



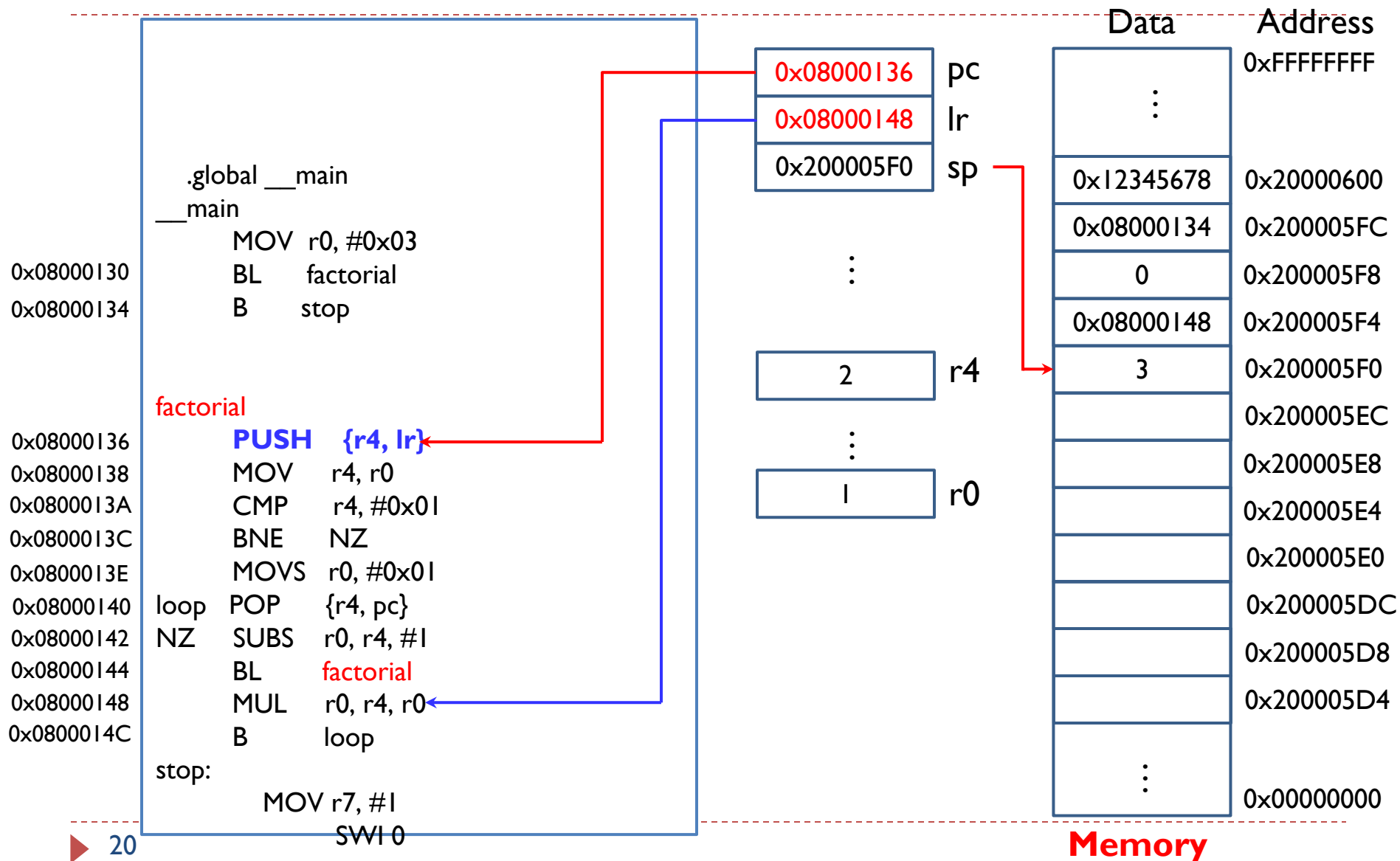
Recursive Factorial in Assembly



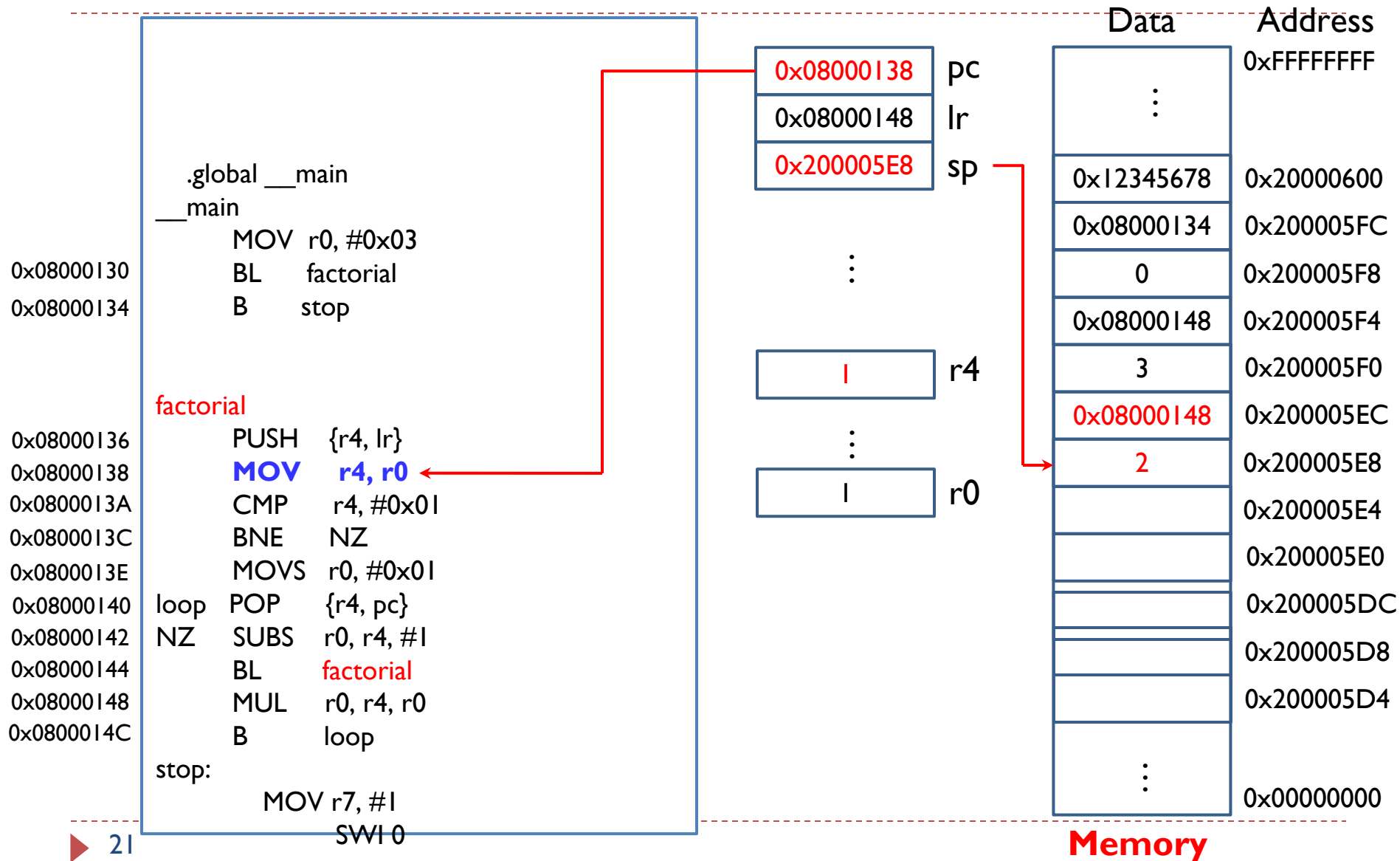
Recursive Factorial in Assembly



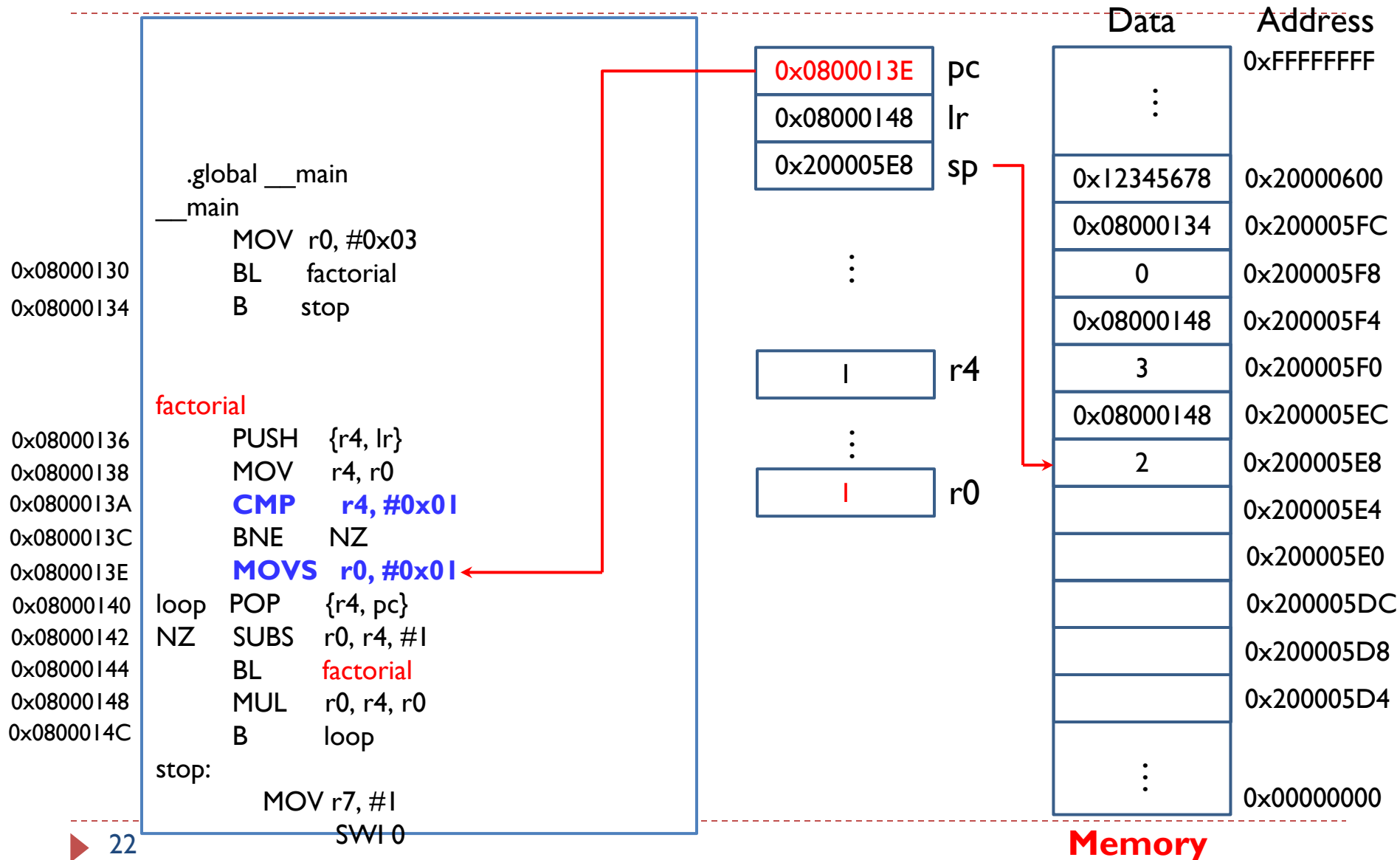
Recursive Factorial in Assembly



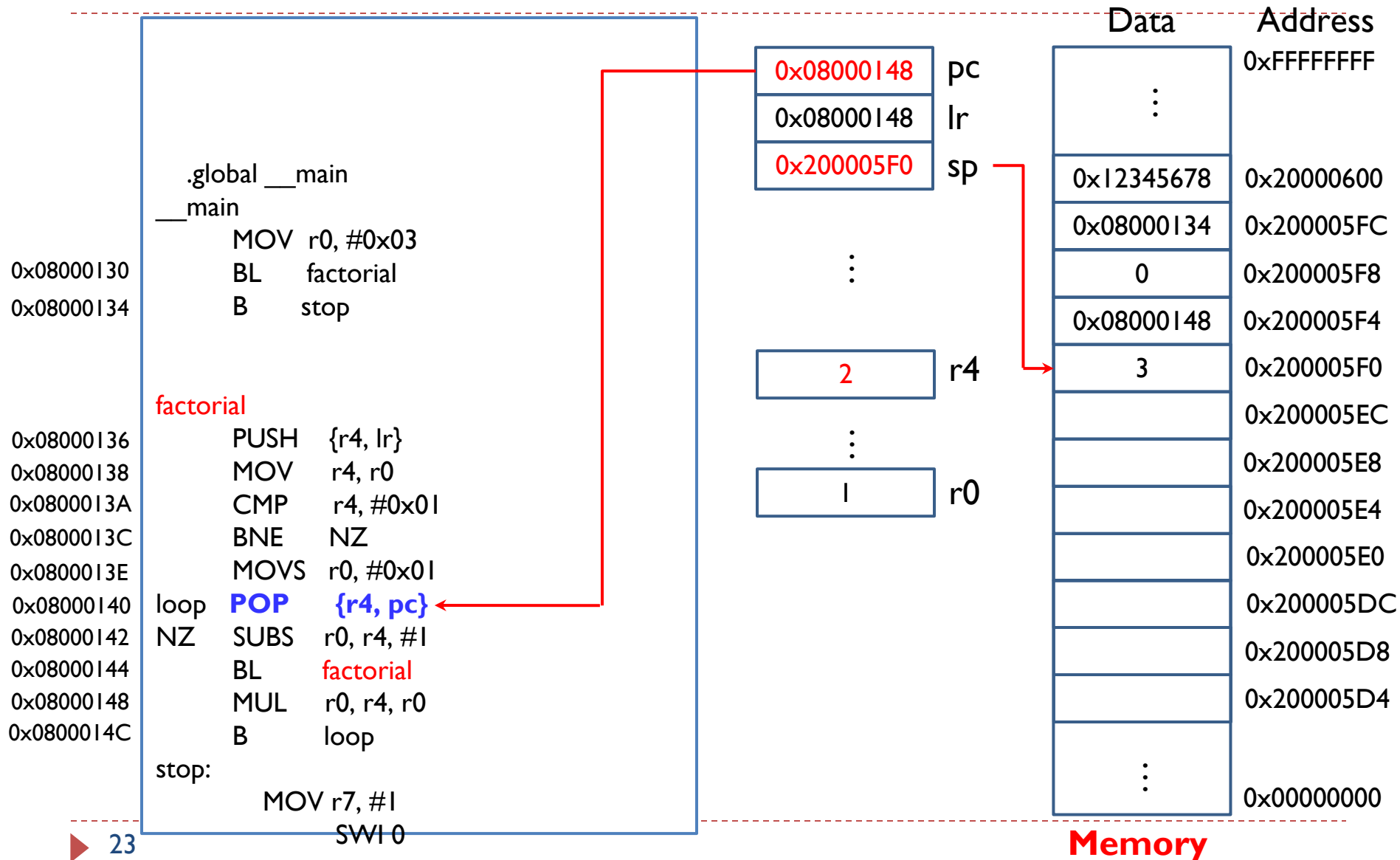
Recursive Factorial in Assembly



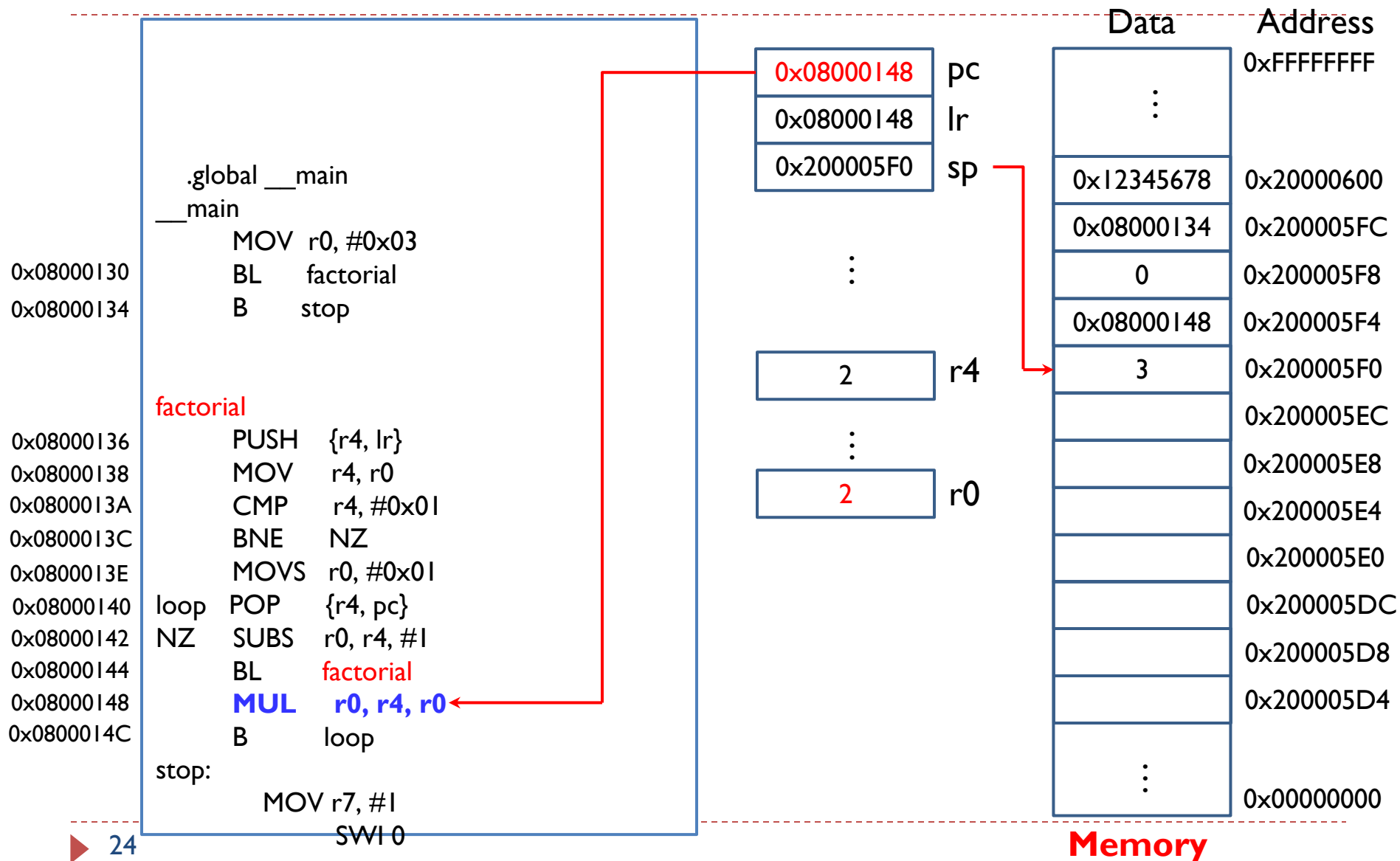
Recursive Factorial in Assembly



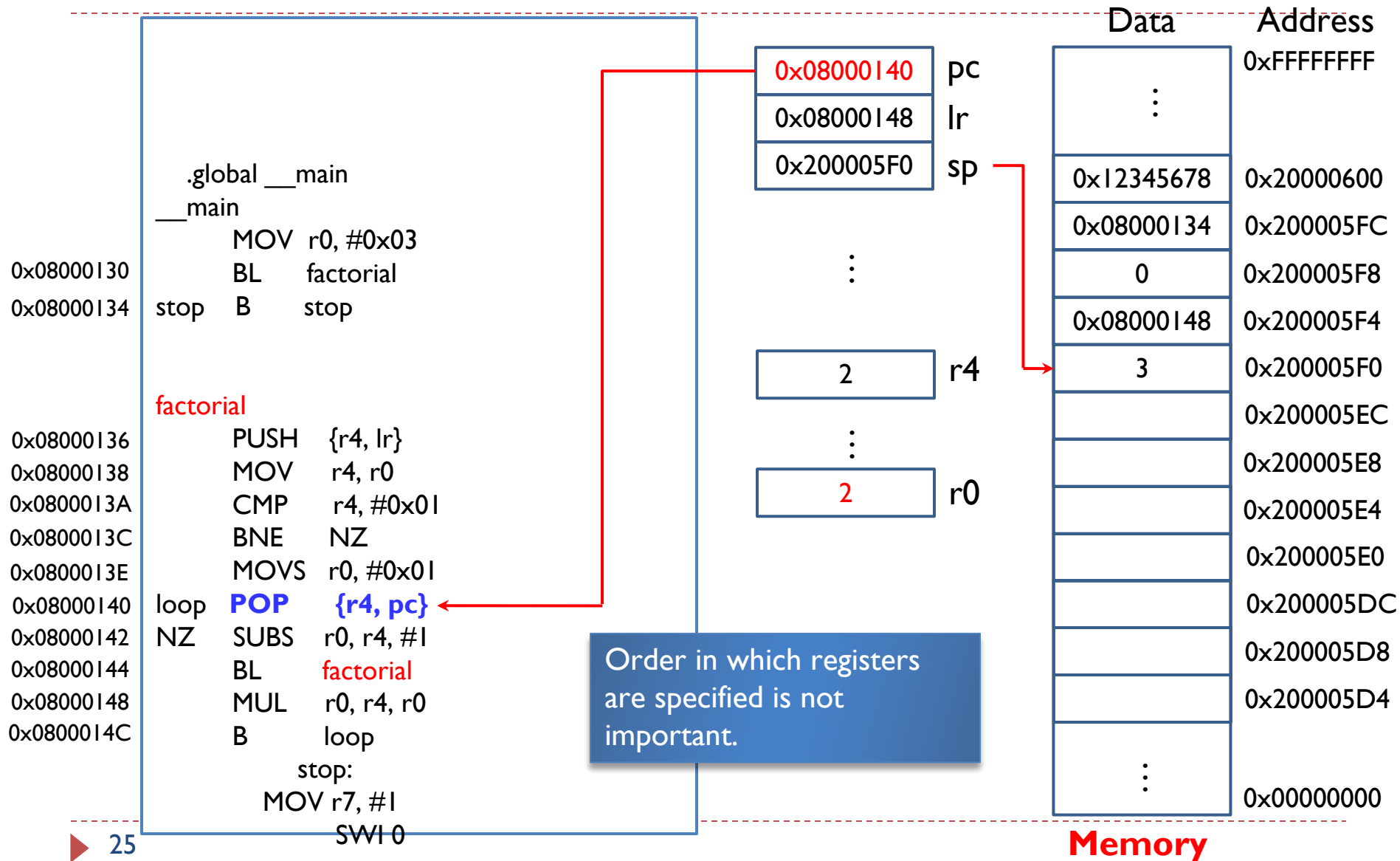
Recursive Factorial in Assembly



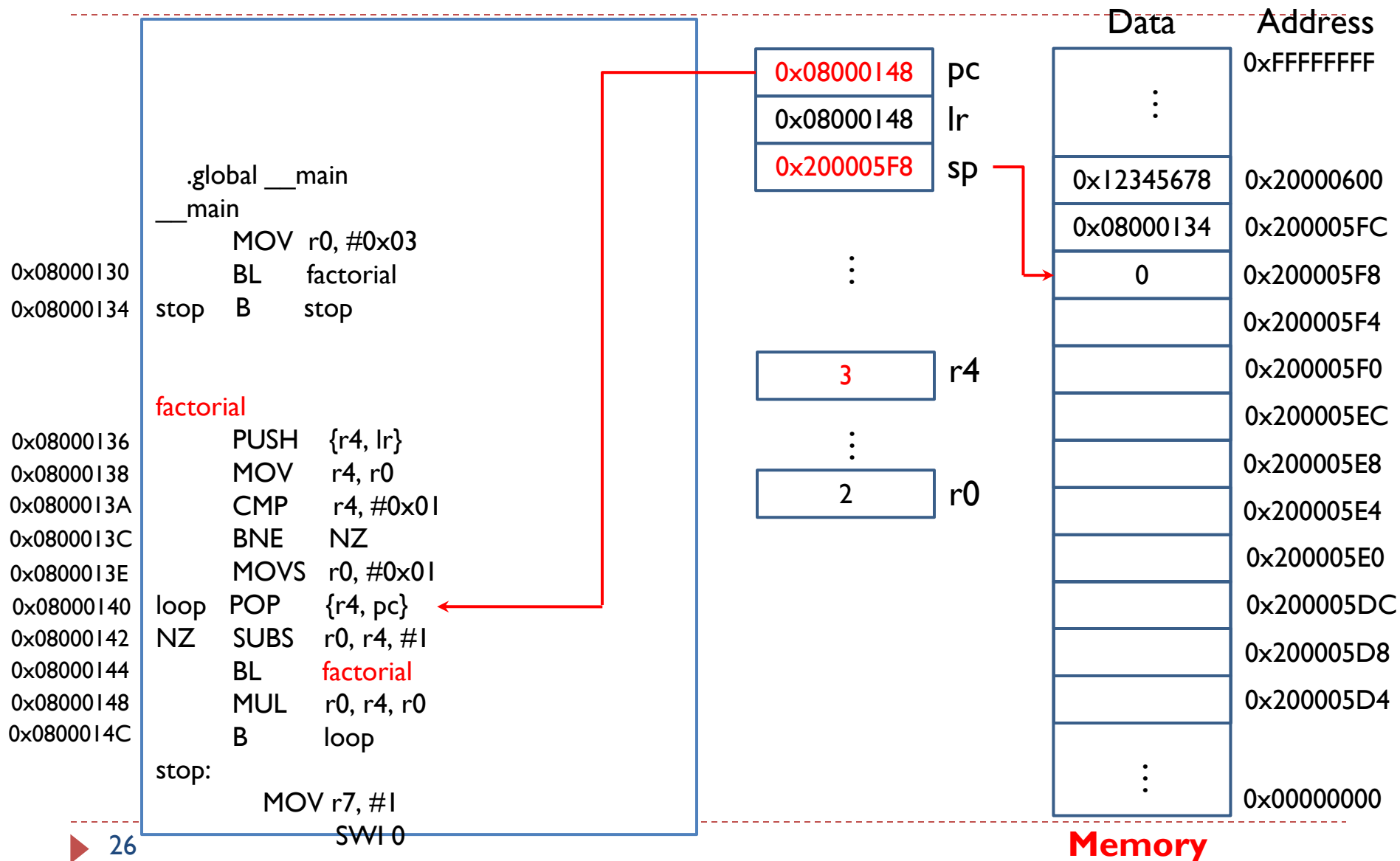
Recursive Factorial in Assembly



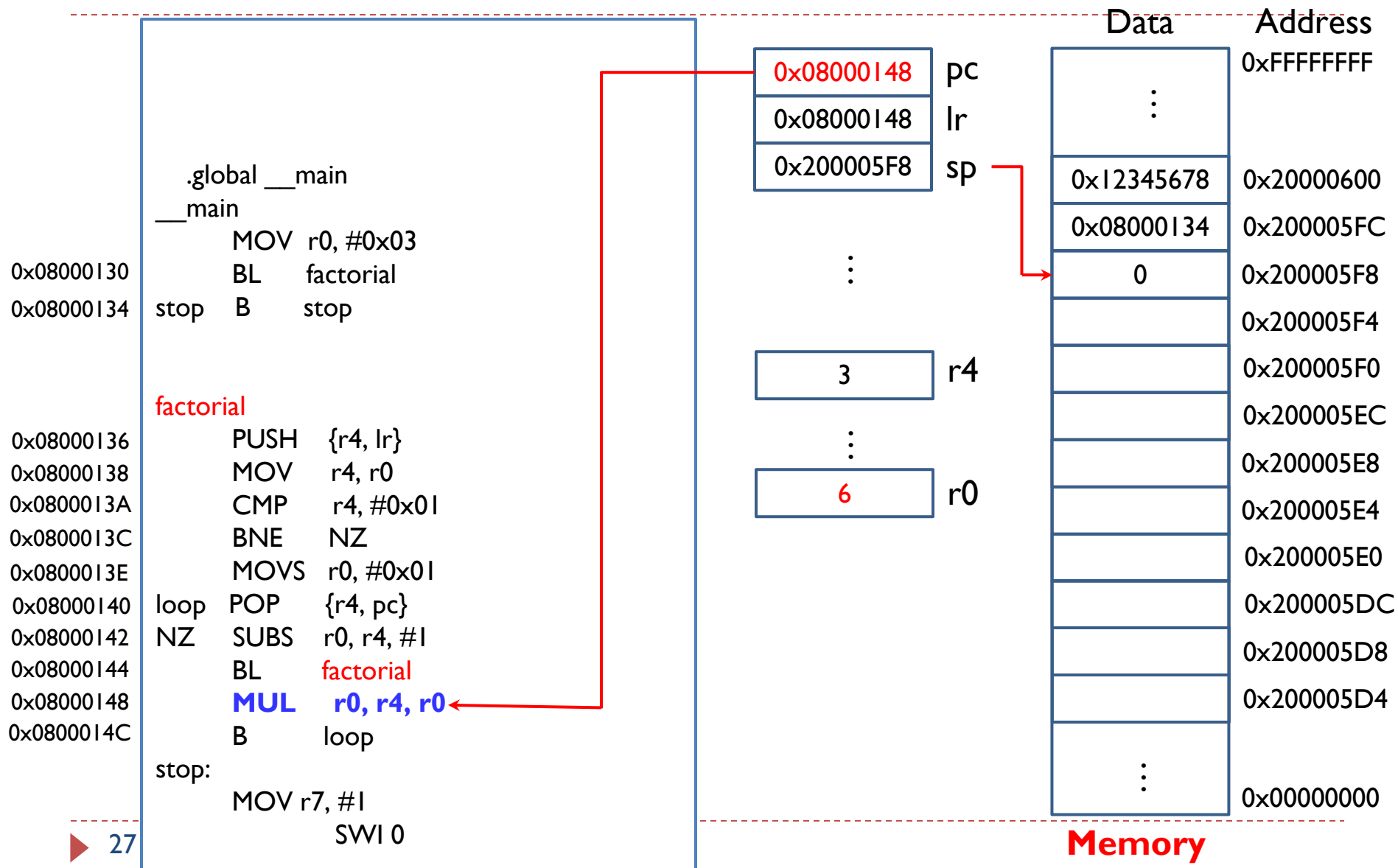
Recursive Factorial in Assembly



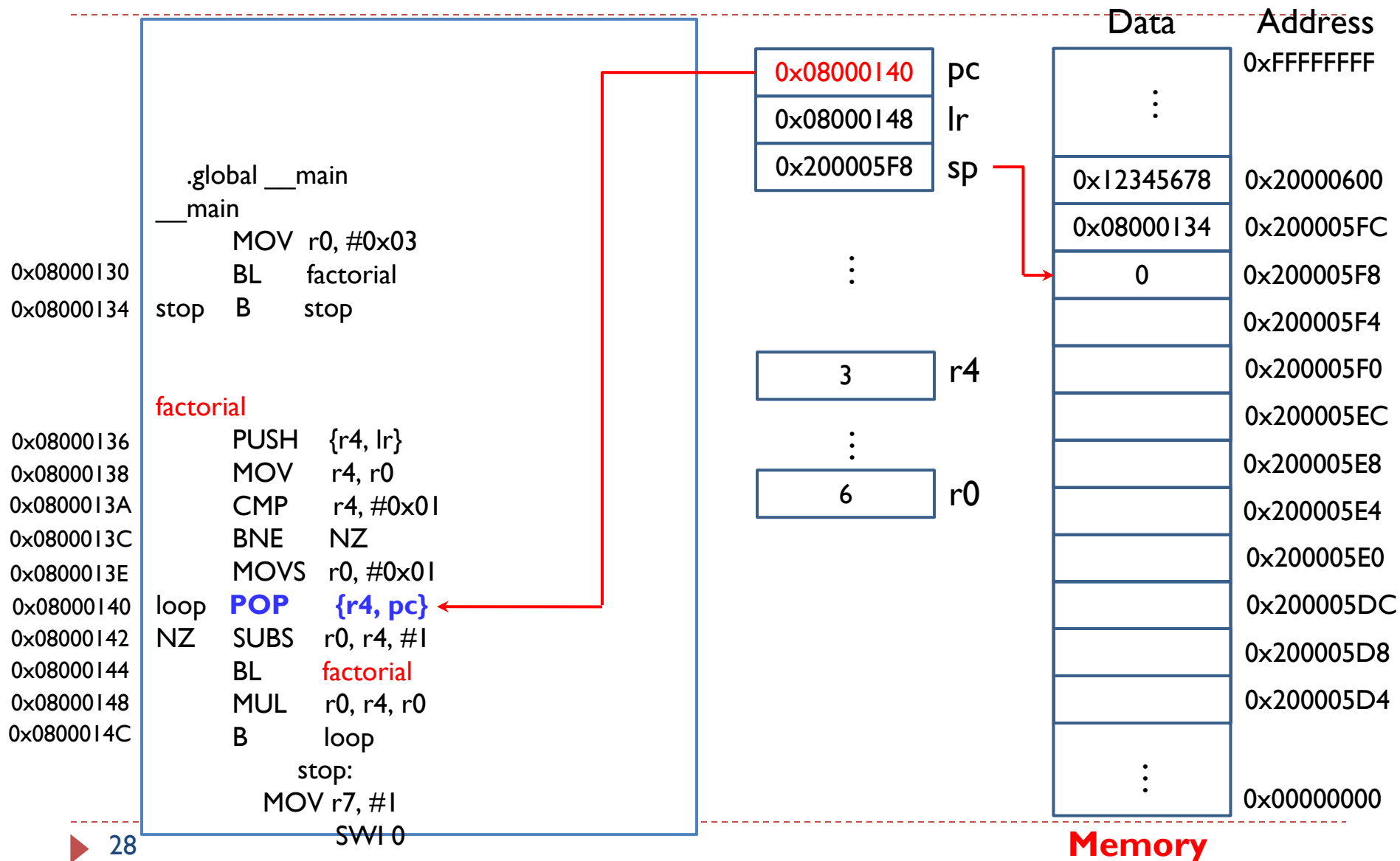
Recursive Factorial in Assembly



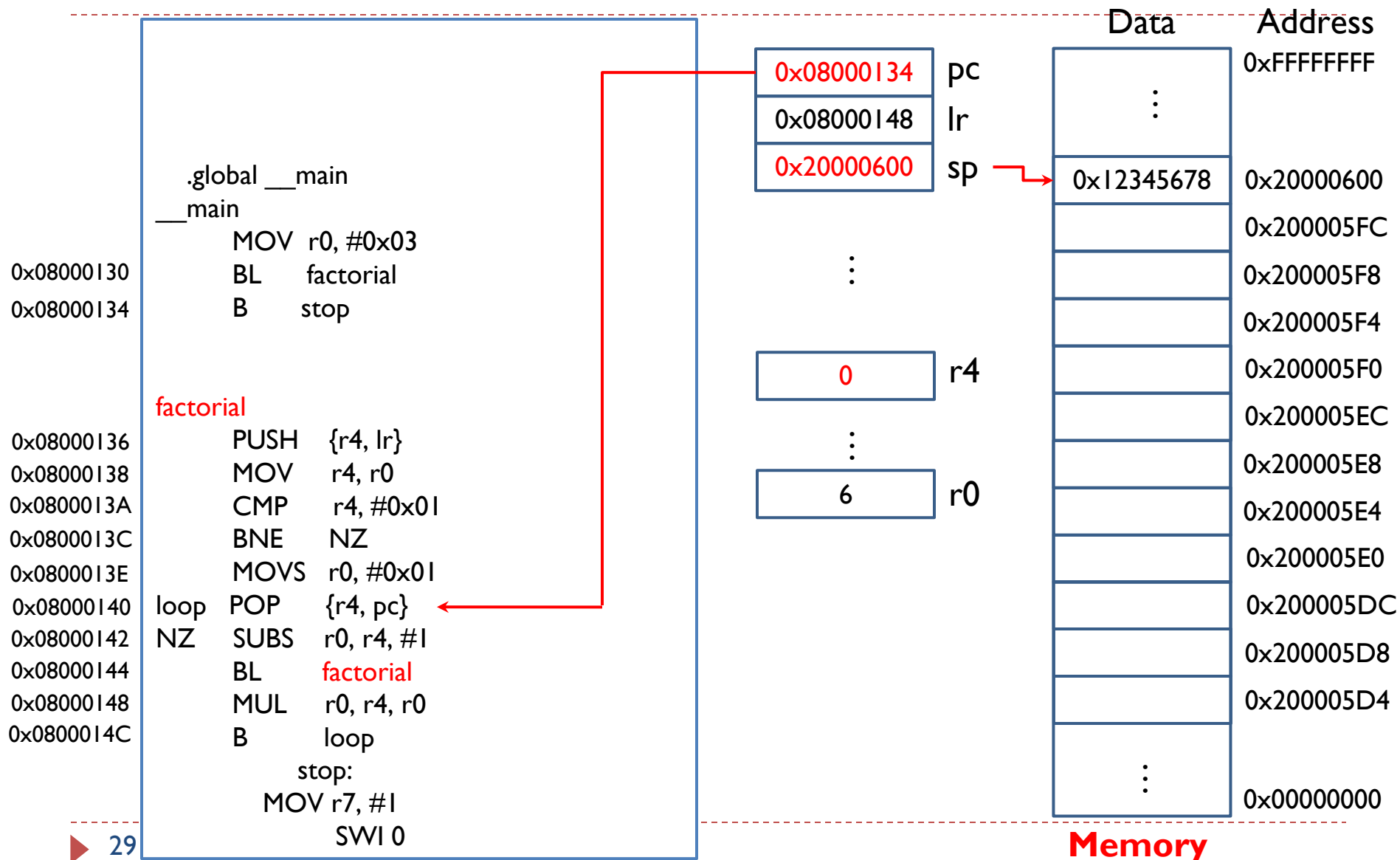
Recursive Factorial in Assembly



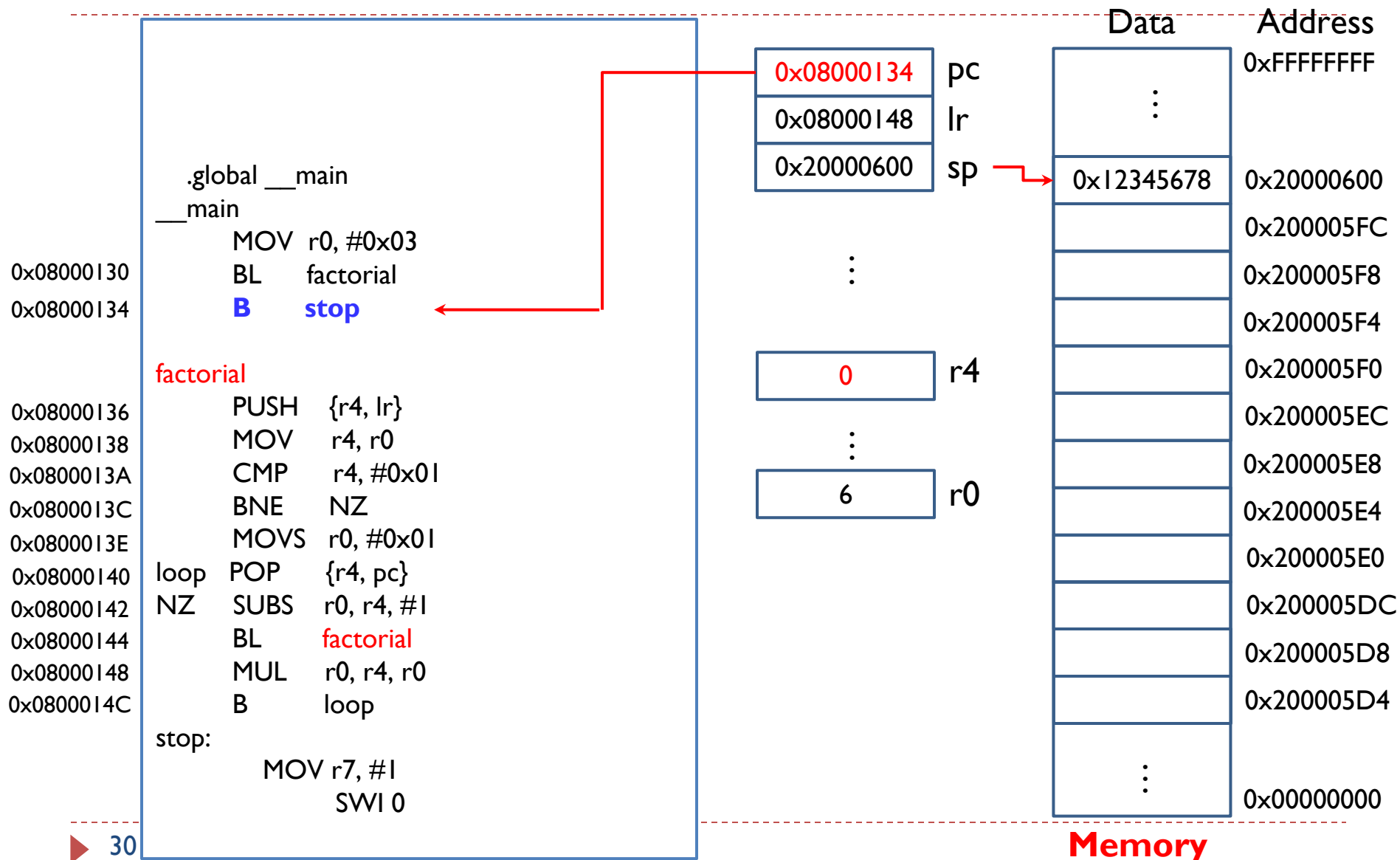
Recursive Factorial in Assembly



Recursive Factorial in Assembly



Recursive Factorial in Assembly



Conclusion

❖ Instructions so far:

❖ Previously:

ADD, SUB, MUL, MULA, [U|S]MULL, [U|S]MLAL, RSB
AND, ORR, EOR, BIC
MOV, MVN
LSL, LSR, ASR, ROR
CMP, B{EQ, NE, LT, LE, GT, GE}
LDR, LDR, STR, LDRB, STRB, LDRH, STRH
BL, BX

❖ New:

PUSH, POP

❖ Registers we know so far

❖ All of them!

❖ Addressing mode so far

❖ Most of them!

